



מבוא למדעי המחשב מ'ח' (234114 \ 234117)

מועד א' אביב תשפ"ג

מבחן מסכם, 31.07.2023

2	3	4	1	1	
---	---	---	---	---	--

רשום/ה לקורס:

--	--	--	--	--	--	--	--	--	--

מספר סטודנט:

משך המבחן: שלוש שעות.

הנחיות כלליות:

- בדקו שיש 8 עמודים (4 שאלות) במבחן, כולל עמוד זה.
- אלא אם כן נאמר אחרת בשאלות, אין להשתמש בפונקציות ספריה או בפונקציות שמומשו בכיתה, למעט פונקציות קלט/פלט והקצאת זיכרון (malloc, free). ניתן להשתמש בטיפוס bool המוגדר ב-stdbool.h.
- אין להשתמש במשתנים סטטיים וגלובליים אלא אם נדרשתם לכך מפורשות.
- הקפידו על סגנון כתיבה כזה שקורא התוכנית ידע לפענח את רעיונותיכם:
 - התוכנית יכולה להכיל תיעוד קל להבנה.
 - על התוכנית להיות כתובה באופן מסודר ומדורג.
 - יש לתת שמות פונקציות ושמות משתנים משמעותיים.
 - ערכים קבועים יש להגדיר בעזרת define.
 - ניתן להוסיף פונקציות עזר כרצונכם.
- נוהל "לא יודע": אם תורידו את ההערה מהקריאה לפונקציה `printIDontKnow()` על שאלה שבה אתם נדרשים לקודד, תקבלו 20% מהניקוד. דבר זה מומלץ אם אתם יודעים שאתם לא יודעים את התשובה.
- המבחן מורכב ממספר רכיבים:
 - תחילה יש לענות על "הצהרה על טוהר הבחינות".
 - לאחר מכן תוכלו לראות את המטלות של המבחן (כל מטלה ברכיב VPL נפרד). פתרו את כל המטלות.
 - כדי להגיש את המבחן יש לענות על "הגשת המבחן". חשוב: על רכיב זה ניתן לענות פעם אחת בלבד.
- שימו לב שקיבלתם את קבצי ה-C ובהם שלד של התוכנית. עליכם רק להשלים את המימוש של הפונקציה הנדרשת.
- לכל שאלה קיבלתם מספר טסטים. מומלץ להוסיף טסטים שלכם.
- לא ניתן לעבוד בסביבת עבודה ייעודית לשפת C שאינה VPL.
- במהלך המבחן לא ייענו שאלות בנוגע לרכיב ה-VPL.



• **נוסחאות שימושיות:**

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \Theta(\log n) \quad \log 1 + \log 2 + \dots + \log n = \Theta(n \log n)$$

$$1^k + 2^k + 3^k + \dots + n^k = \Theta(n^{k+1}) \quad 1 + k + k^2 + k^3 + \dots + k^n = \Theta(k^n), \quad k > 1$$

$$S_n = \frac{a_1}{1-q} \text{ אינסופית: } S_n = a_1 * \frac{q^n - 1}{q - 1}$$

$$S_n = n * \frac{a_1 + a_n}{2} \text{ סכום סדרה חשבונית:}$$

צוות הקורס 234114/7

מרצים: פרופ' מירלה בן חן (מרצה אחראית), גב' יעל ארז; **מתרגלים:** אמיר אלקלעי (מתרגל אחראי), שלי גולן (מתרגלת אחראית), אדיר רחמים, דניאל אלגריסי, עמר צברי, רותם אליאס, תמיר שור.



בשאלה 1 בחרו את האות המייצגת את הסיבוכיות המתאימה מתוך טבלת הפונקציות להלן:

- | | |
|---|---|
| a. $\Theta(1)$ | n. $\Theta(n^3 \log n)$ |
| b. $\Theta(\log \log n)$ | o. $\Theta(n^3)$ |
| c. $\Theta(\log n)$ | p. $\Theta(n^4)$ |
| d. $\Theta(\log^2 n)$ | q. $\Theta(2^{\sqrt{n}})$ |
| e. $\Theta(\sqrt{n})$ | r. $\Theta\left(2^{\frac{n}{3}}\right)$ |
| f. $\Theta(\sqrt{n} \log n)$ | s. $\Theta\left(2^{\frac{n}{2}}\right)$ |
| g. $\Theta(n)$ | t. $\Theta\left(3^{\frac{n}{3}}\right)$ |
| h. $\Theta(n \log n)$ | u. $\Theta\left(3^{\frac{n}{2}}\right)$ |
| i. $\Theta(n \log^2 n)$ | v. $\Theta(2^n)$ |
| j. $\Theta\left(n^{\frac{3}{2}}\right)$ | w. $\Theta(n \cdot 2^n)$ |
| k. $\Theta(n^2)$ | x. $\Theta(3^n)$ |
| l. $\Theta(n^2 \log(\log(n)))$ | y. $\Theta(n^2 3^n)$ |
| m. $\Theta(n^2 \log n)$ | z. $\Theta(n^3 3^n)$ |

מה שעליכם לעשות הוא לשנות את האות "a" לאות המתאימה לתשובה שלכם בכל סעיף. שימו לב שלכל סעיף יש לבחור גם את האות המתאימה לסיבוכיות הזמן (time complexity) וגם את האות המתאימה לסיבוכיות המקום (space complexity).

**שאלה 1: [25 נקודות]**

א. חשבו את סיבוכיות הזמן והמקום של הפונקציה f1 המוגדרת בקטע הקוד הבא, כפונקציה של n. אין צורך לפרט את שיקוליכם. חובה לפשט את הביטוי ככל שניתן.

```
void f1(int n) {
    int a = 0;
    for (int j = n, i = 0; i < n; i++, j *= n) {
        for (int k = j; k > 0; k /= 2);
        a += n;
    }
    free(malloc(sizeof(int) * a));
}
```

ב. חשבו את סיבוכיות הזמן והמקום של הפונקציה f2 המוגדרת בקטע הקוד הבא, כפונקציה של n. אין צורך לפרט את שיקוליכם. חובה לפשט את הביטוי ככל שניתן.

```
int g(int n){
    int m = 2;
    for (int i = 0; i < n; i++) {
        m *= 2;
    }
    return m;
}

void f2(int n) {
    for (int i = 1; i < n; i *= 2) {
        int k = g(i);
        while (k) {
            k /= 2;
        }
    }
}
```



ג. חשבו את סיבוכיות הזמן והמקום של הפונקציה $f3$ המוגדרת בקטע הקוד הבא, כפונקציה של n . אין צורך לפרט את שיקוליכם. חובה לפשט את הביטוי ככל שניתן.

```
int aux(int n) {
    int b = 4;
    while (n) {
        b *= 4;
        n /= 2;
    }
    free(malloc(b));
    return b;
}

void f3(int n){
    if (n <= 1) {
        return;
    }
    for (int i = 0; i < 4; i++) {
        aux(n);
    }
    f3(n / 2);
}
```



שאלה 2 (25 נקודות):

ממשו פונקציה שחתימתה:

```
int q2(char * arr[], int n);
```

הפונקציה מקבלת מערך מחרוזות באורך n . ידוע כי אורכה של המחרוזת הראשונה קטן מאורכה של המחרוזת האחרונה. הפונקציה תחזיר אינדקס שבו מחרוזת ארוכה מזו שלפניה. ובניסוח מתמטי: אינדקס i - במערך עבורו מתקיים: $strlen(arr[i - 1]) < strlen(arr[i])$

להלן מספר דוגמאות:

המערך	הערך שהפונקציה תחזיר	הסבר
<code>{"a", "ab", "cc", "abcd"}</code>	1 או 3	המחרוזת באינדקס 1 אורכה 2, שזה יותר מאורכה של המחרוזת באינדקס 0. המחרוזת באינדקס 3 אורכה 4, שזה יותר מאורכה של המחרוזת באינדקס 2.
<code>{"zzz", "aaaaaaa"}</code>	1	המחרוזת באינדקס 1 אורכה 7, שזה יותר מאורכה של המחרוזת באינדקס 0.
<code>{"abc", "az", "aaaaaaa"}</code>	2	המחרוזת באינדקס 2 אורכה 7, שזה יותר מאורכה של המחרוזת באינדקס 1.

- נתונה לכם תוכנית אשר קוראת מהשתמש את אורך המערך, ולאחר מכן את המחרוזות, קוראת לפונקציה ומדפיסה למסך מה שהפונקציה מחזירה.
- ניתן להניח כי הקלט חוקי.
- ניתן להניח כי אורכן של המחרוזות אינו עולה על 30 תווים.

דרישות סיבוכיות:

- סיבוכיות זמן: $\Theta(\log n)$
- סיבוכיות מקום נוסף: $\Theta(1)$



שאלה 3 (25 נקודות):

ממשו פונקציה שחתימתה:

$$\text{int } q3(\text{int array}[], \text{int } n);$$

הפונקציה מקבלת מערך של מספרים שלמים באורך n . מצאו את אורך תת המערך הקטן ביותר כך שאם נמין אותו – נקבל שהמערך ממוין בשלמותו.

להלן מספר דוגמאות:

המערך	הערך שהפונקציה תחזיר	הסבר
{2,6,4,8,10,9,15}	5	תת המערך שנרצה למיין הוא: {6,4,8,10,9} באורך 5. לאחר מכן נקבל כי המערך כולו ממוין בשלמותו.
{1,2,3,4}	0	המערך כבר ממוין.
{3,4,7,6}	2	תת המערך שנרצה למיין הוא {7,6} ואורכו 2. לאחר מכן נקבל כי המערך כולו ממוין בשלמותו.

- נתונה לכם תוכנית אשר קוראת מהמשתמש את אורך המערך, ולאחר מכן את תוכן המערך, קוראת לפונקציה ומדפיסה למסך מה שהפונקציה מחזירה.
- ניתן להניח כי הקלט חוקי.

דרישות סיבוכיות:

- סיבוכיות זמן: $\Theta(n \log n)$
- סיבוכיות מקום נוסף: $\Theta(n)$



שאלה 4 Backtracking (25 נקודות):

ממשו פונקציה שחתימתה:

```
int q4(int array[], int n, int target);
```

הפונקציה מקבלת מערך של מספרים שלמים חיוביים באורך n , ומספר שלם חיובי $target$.

חלוקה חוקית של המערך, היא חלוקת אברי המערך לתתי קבוצות, כך ש:

1. סכום האברים בכל תת קבוצה הוא $target$.

2. כל אבר במערך מופיע **לפחות** בתת קבוצה אחת, ולכל **היותר** בשתי תתי קבוצות.

על הפונקציה להחזיר את מספר הקבוצות **המינימלי** בחלוקה חוקית כלשהיא של המערך. אם אין אף חלוקה חוקית יש להחזיר -1 .

להלן מספר דוגמאות:

המערך	ערך $target$	הערך שהפונקציה תחזיר	הסבר
{2,3,7,5}	7	-1	לא קיימת חלוקה חוקית עם סכום 7
{1,2,3,4}	4	3	החלוקה החוקית היחידה היא: {1,3}{2,2}{4}
{3,4,7,6}	10	2	החלוקה החוקית היחידה היא: {3,7}{4,6}

הערות:

- יש להשתמש בשיטת backtracking כפי שנלמדה בכיתה.
- בשאלה זו אין דרישת סיבוכיות, אולם כמקובל ב backtracking - יש לוודא שלא מתבצעות קריאות רקורסיביות מיותרות עם פתרונות שאינם חוקיים.
- ניתן ומומלץ להשתמש בפונקציות עזר.
- נתונה לכם תוכנית אשר קוראת מהמשתמש את אורך המערך, ולאחר מכן את תוכן המערך, ולבסוף את המספר $target$, קוראת לפונקציה ומדפיסה למסך מה שהפונקציה מחזירה.

בהצלחה!